LAPRAK TEKNOLOGI IOT

TUGAS PRAKTIKUM MODUL 6

Oleh:

NAMA : ERVAN GUSTI WANJAYA

NPM : 25782096

KELAS : TRI – 3D

Dosen:

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd.,M.Kom.

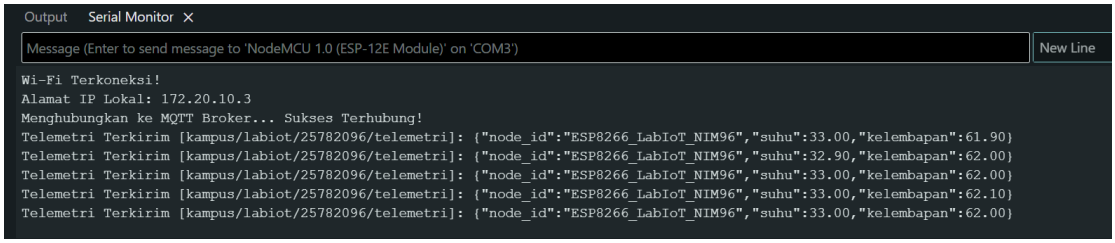
TEKNOLOGI REKAYASA INTERNET

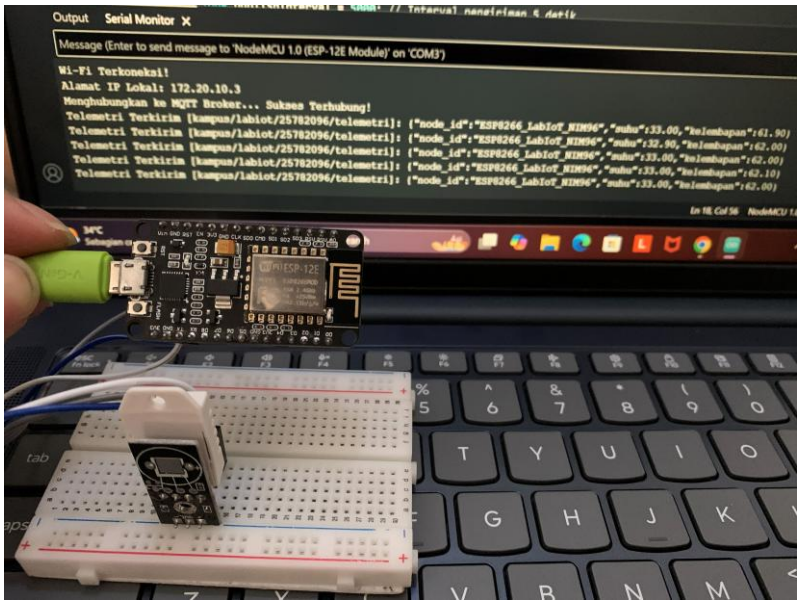
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

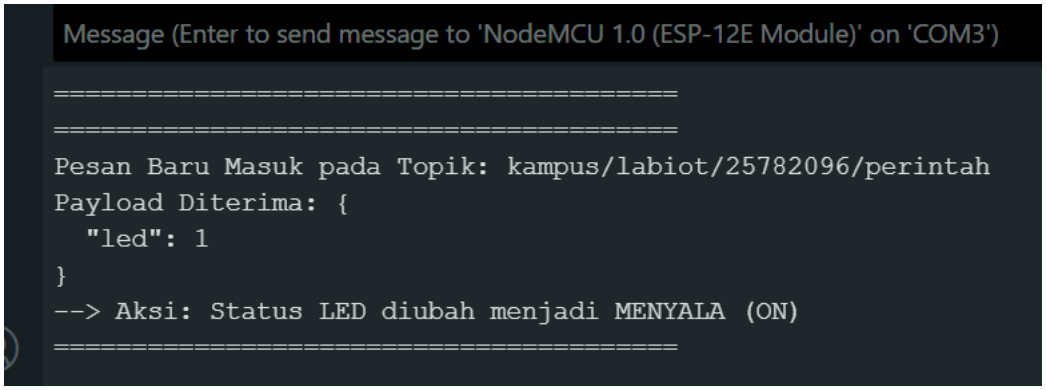
Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

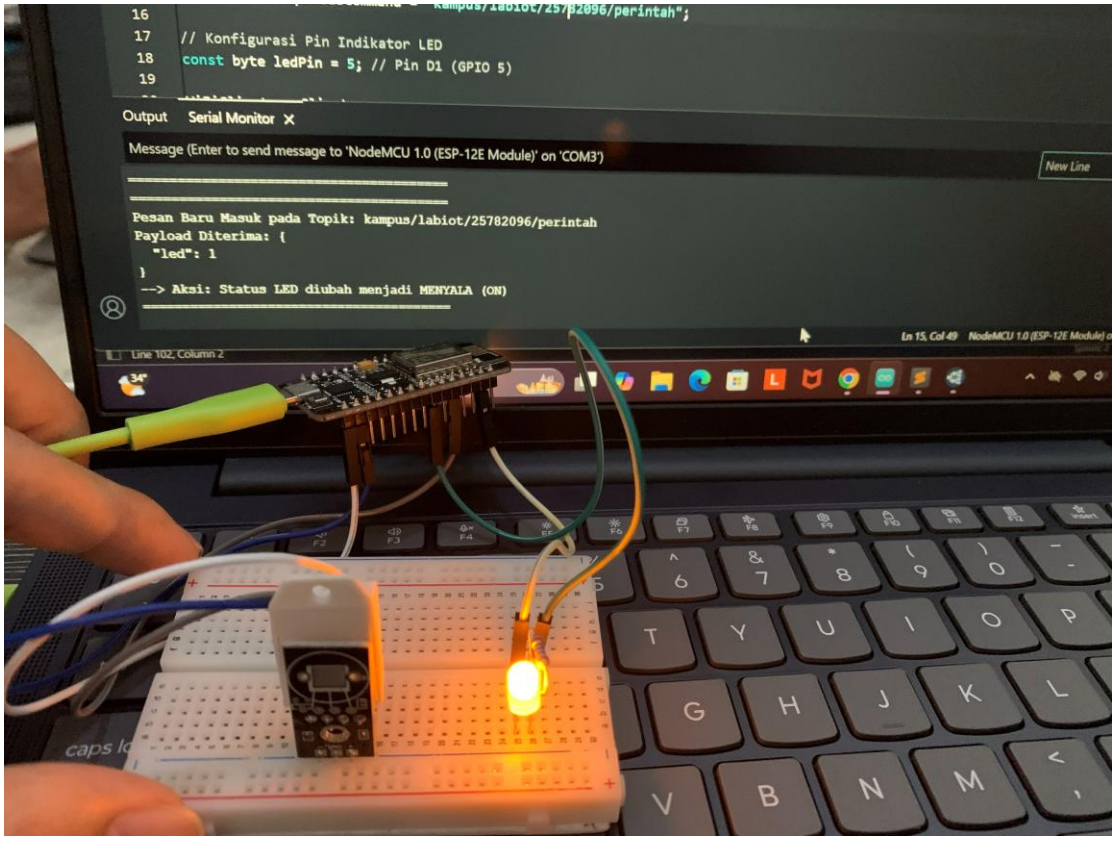




Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

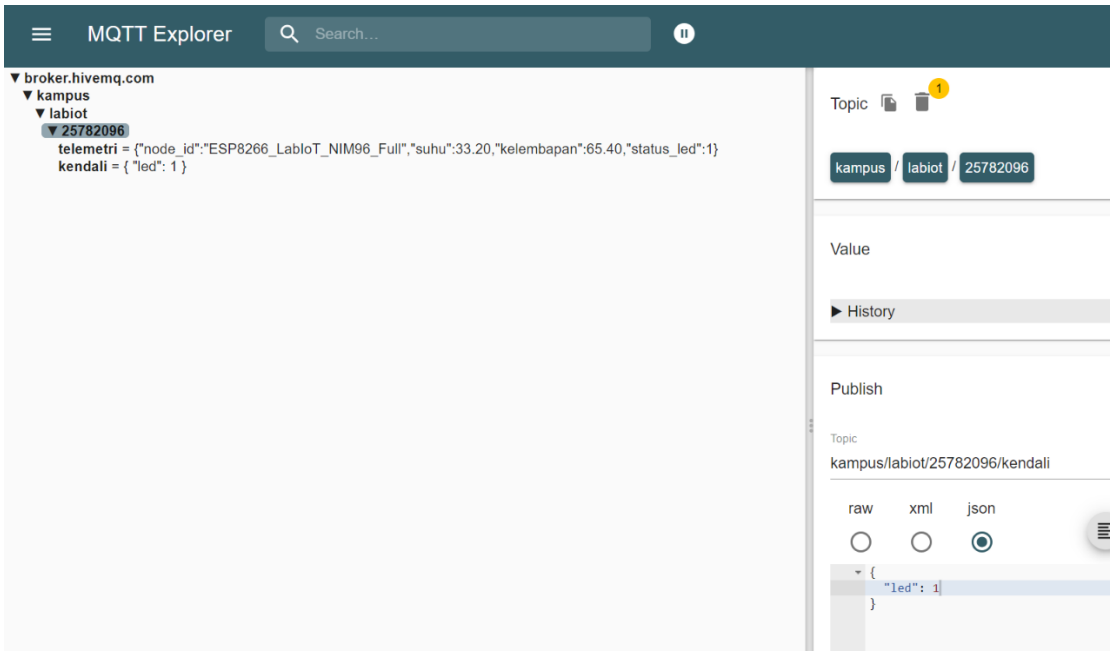
```
Mencoba koneksi MQTT...Gagal (rc=-2). Ulangi dalam 5 detik...
Mencoba koneksi MQTT...Gagal (rc=-2). Ulangi dalam 5 detik...
Mencoba koneksi MQTT...Gagal (rc=-2). Ulangi dalam 5 detik...
Mencoba koneksi MQTT...Gagal (rc=-2). Ulangi dalam 5 detik...
Mencoba koneksi MQTT...Gagal (rc=-2). Ulangi dalam 5 detik...
Mencoba koneksi MQTT...Gagal (rc=-2). Ulangi dalam 5 detik...
Mencoba koneksi MQTT... Terhubung!
Subskripsi Topik: kampus/labiot/25782096/perintah
```

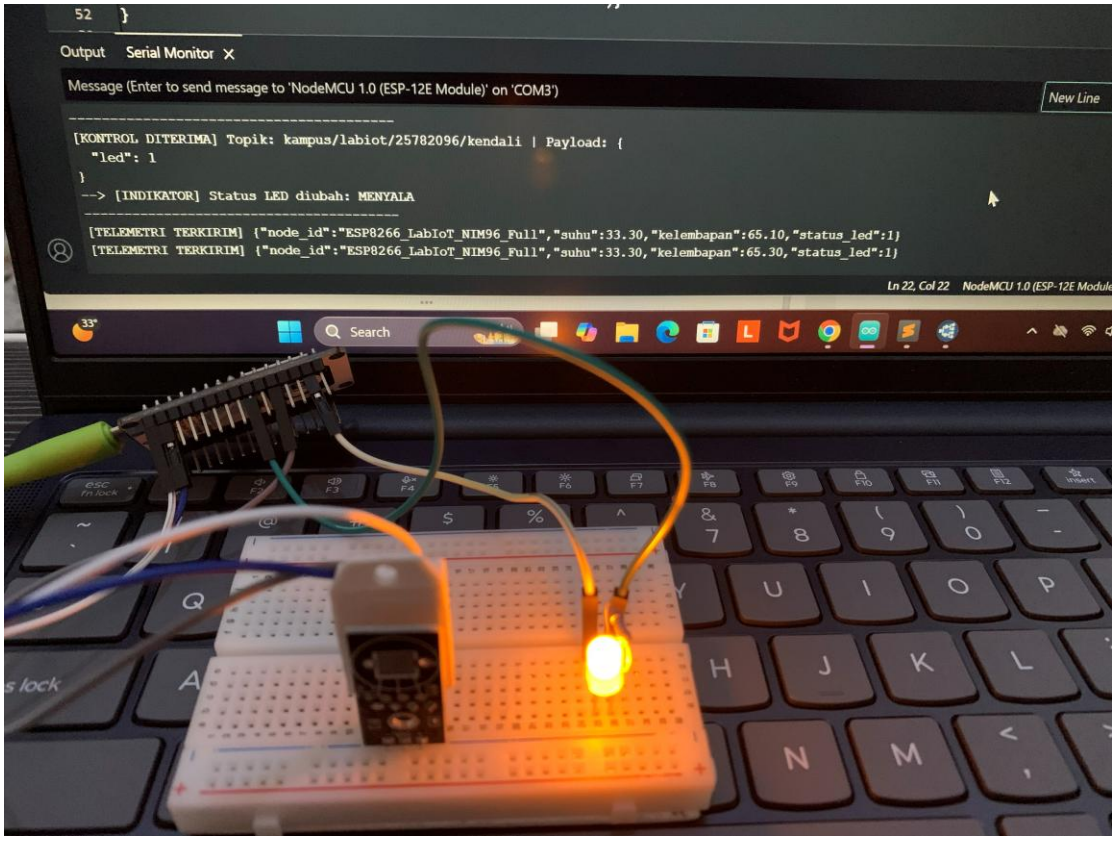




Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

```
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.40,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.30,"kelembapan":65.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.40,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.40,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM96_Full","suhu":33.20,"kelembapan":65.40,"status_led":0}
```





6.6 Latihan

Latihan 1: Analisis Arsitektur & Perbandingan Protokol

1. **Efisiensi Bandwidth pada Paket Data Seluler:**

    - **HTTP:** Memiliki beban *header* berbasis teks yang besar ($\approx 200 - 800$ byte) pada setiap *request/response*. Model *polling* berulang membuang banyak data karena *header* wajib dikirim secara utuh meskipun data sensor tidak berubah.
    - **MQTT:** Menggunakan *header* biner yang sangat ringkas (minimal 2 byte). Model *Publish-Subscribe* hanya mengirimkan paket data saat terjadi perubahan atau sesuai interval (*push*), sehingga sangat efisien dalam penggunaan kuota data seluler.

2. **Beban Komputasi Mikrokontroler (*CPU Overhead*):**

    - **HTTP:** Mikrokontroler terbeban oleh proses penguraian (*parsing*) *string header* HTTP yang panjang serta harus membuka dan menutup koneksi TCP secara berulang, yang menguras memori SRAM dan siklus CPU.
    - **MQTT:** Menggunakan satu saluran koneksi TCP persisten (*persistent connection*). Penguraian biner dilakukan secara asinkron lewat fungsi *callback*, sehingga beban pemrosesan CPU mikrokontroler menjadi jauh lebih ringan.

3. **Kemampuan Distribusi Data Simultatif (*1-to-N Distribution*):**

    - **HTTP:** Berbasis koneksi *point-to-point*. Jika satu pengirim ingin mendistribusikan data ke N penerima, pengirim harus membuka N koneksi terpisah secara bergantian (beban komputasi $O(N)$).
    - **MQTT:** Terpusat menggunakan *Broker*. *Publisher* cukup mengirimkan data 1 kali ke *Broker*, lalu *Broker* yang bertugas menyebarkan data tersebut ke N *Subscriber* yang berlangganan topik tersebut secara bersamaan.

Latihan 2: Analisis Hierarki Topik & Karakter Wildcard

Struktur topik: pabrik/<line>/<mesin>/<sensor>

1. Memantau parameter suhu dari seluruh mesin di semua lini: **pabrik/+/+/suhu**
2. Memantau seluruh parameter sensor khusus mesin m1 di lini line2: **pabrik/line2/m1/#**
3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan mesin: **pabrik/#**

Latihan 3: Implementasi Serialisasi Data Dinamis

```
// Alokasi dokumen JSON statis
StaticJsonDocument<256> doc;
doc["node_id"] = clientId;
doc["suhu"] = serialized(String(t, 2));
doc["kelembapan"] = serialized(String(h, 2));

// --- MODIFIKASI LATIHAN 3 ---
doc["uptime_detik"] = millis() / 1000;
doc["ip_lokal"] = WiFi.localIP().toString();
// ----------------------------

char jsonBuffer[256];
serializeJson(doc, jsonBuffer);
```

Latihan 4: Analisis Mekanisme Reconnect & Loop Blocking

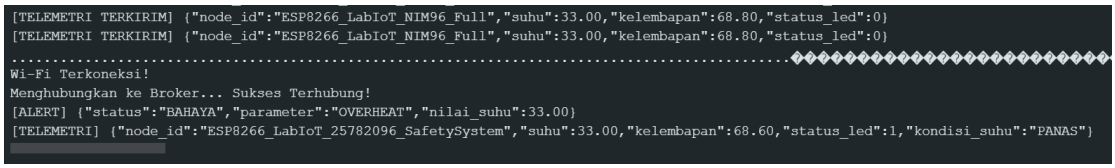
1. **Konsekuensi Menyisipkan delay(6000) di Fungsi loop():**

    - **Penurunan Responsivitas Callback:** Rutin client.loop() tertahan selama 6 detik, menyebabkan perintah kendali LED yang dikirim dari *Broker* mengalami *delay* dan tidak langsung dieksekusi.
    - **Koneksi Terputus (*Keep-Alive Timeout*):** Jika interval *keep-alive ping* MQTT terlewati akibat tertahan delay(), *Broker* akan menganggap mikrokontroler mati dan memutus koneksi TCP secara sepihak.

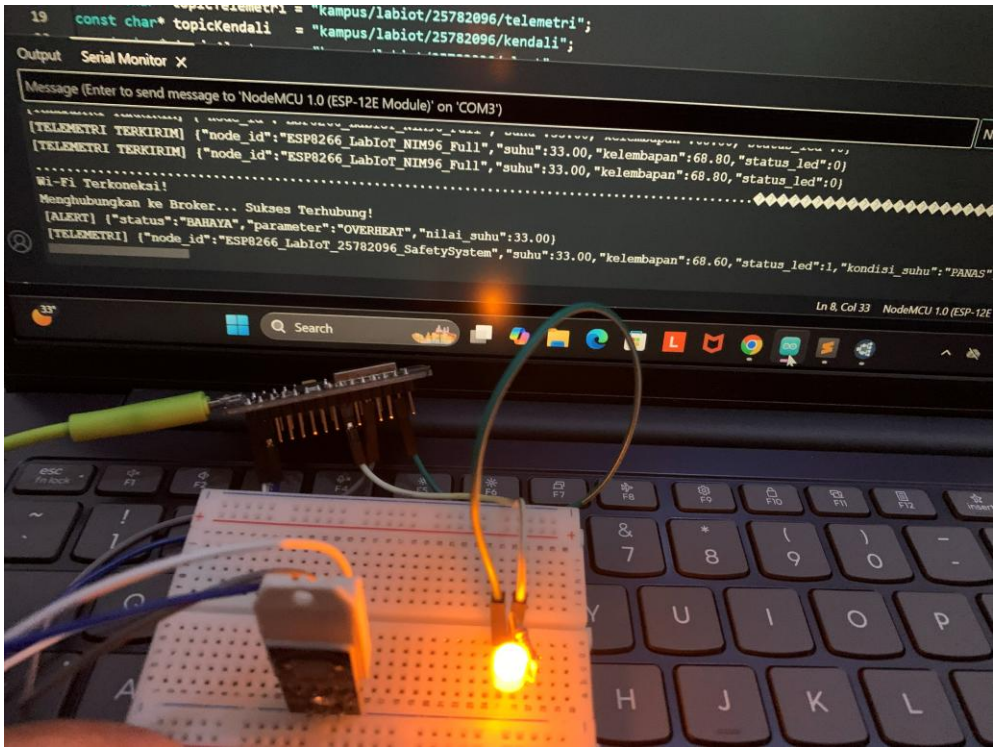
2. **Cara Kerja Fungsi reconnect():**

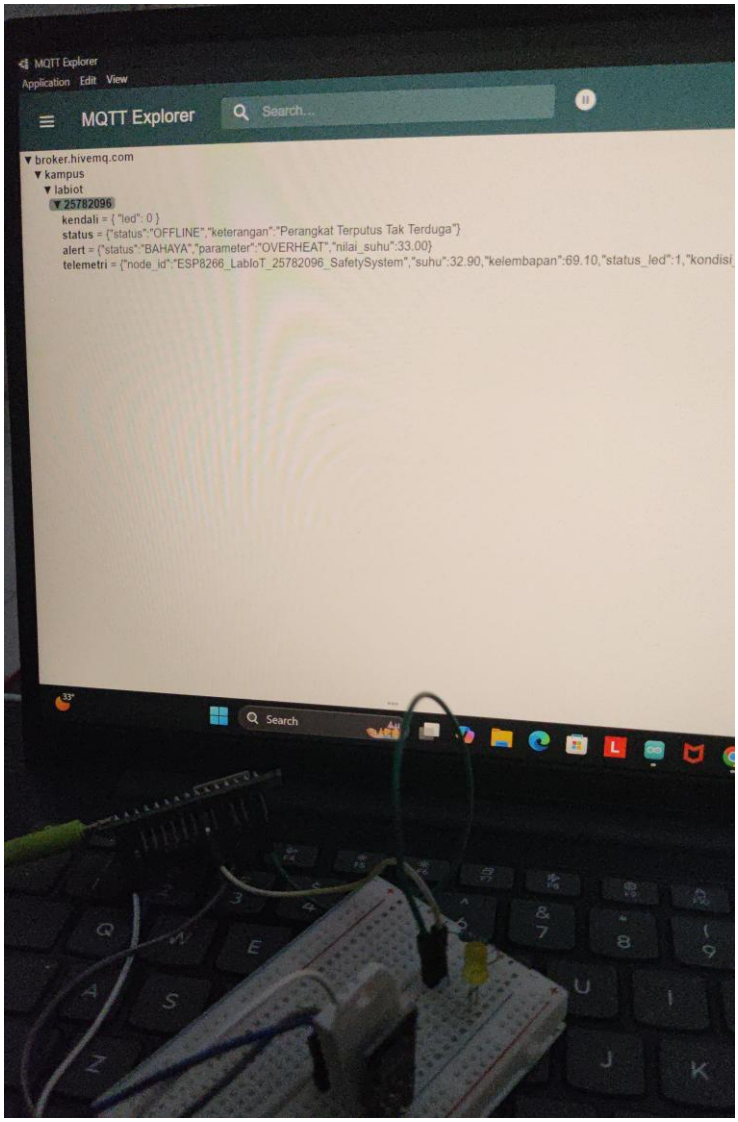
- Fungsi memeriksa status jaringan melalui client.connected().
- Jika bernilai false, fungsi masuk ke perulangan while (!client.connected()) dan mencoba melakukan jabat tangan ulang dengan *Broker* menggunakan client.connect(clientId).
- Setelah koneksi pulih, fungsi mendaftarkan kembali subskripsi topik (client.subscribe(...)) agar mikrokontroler siap menerima pesan kembali tanpa harus di-*reset*.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri








MQTT Explorer
Application Edit View
MQTT Explorer
Search...
▼ broker.hivemq.com
▼ kampus
▼ labiot
▼ 25782096
kendali = { "led": 0 }
status = {"status":"OFFLINE","keterangan":"Perangkat Terputus Tak Terduga"}
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":33.00}
telemetri = {"node_id":"ESP8266_LabIoT_25782096_SafetySystem","suhu":32.90,"kelembapan":69.10,"status_led":1,"kondisi
Search